

Able to Select appropriate NoSQL databases for different application needs, Design efficient NoSQL data models, Perform CRUD operations in MongoDB with aggregation and sharding, Configure Cassandra clusters with tunable consistency and Query and analyze graph data using Neo4j and Cypher	Unit V: NoSQL Databases 5.1 Types of NoSQL Databases (Document, Key-Value, Column-family, Graph) 5.2 NoSQL data Models 5.3 CRUD, Aggregation, Sharding operation and its implementation using NoSQL database tools (<u>MangoDB</u>, <u>Cassandra</u> etc) 5.4 Graph Traversal and Graph databases and its implementation	7 <u>hrs</u>
--	--	----------------------------

Yuba Raj Devkota

NCIT

Subject: Database Engineering (ME CE/MSC CS)

Introduction to NoSQL Database

- *NoSQL (Not Only SQL) is a category of non-relational database systems designed to store, manage, and retrieve large volumes of data that do not fit well into traditional relational (table-based) databases.*
- *Unlike relational databases (RDBMS), NoSQL databases do not rely on fixed schemas and usually avoid complex joins.*
 - Relational DB (RDBMS):
Best for structured data like *student records, banking systems.*
 - NoSQL DB:
Best for unstructured/semi-structured data like *social media posts, product catalogs, logs.*

Common NoSQL data models include:

- Key–Value stores (e.g., Redis)
- Document databases (e.g., MongoDB)
- Column-family stores (e.g., Cassandra)
- Graph databases (e.g., Neo4j)

Why is NoSQL needed?

NoSQL databases emerged to address limitations of traditional RDBMS in modern, large-scale applications.

1. Scalability

- RDBMS mainly scale **vertically** (stronger single machine).
- NoSQL supports **horizontal scaling** (adding more servers easily), ideal for distributed systems.

2. Handling Big Data

- Modern applications generate **huge volumes of data** (logs, social media, sensor data).
- NoSQL efficiently handles **high-volume, high-velocity, and high-variety** data.

3. Flexible Schema

- RDBMS requires a predefined schema.
- NoSQL allows **schema-less or dynamic schemas**, useful when data structure changes frequently.

4. High Performance

- NoSQL databases are optimized for **fast read/write operations**.
- Suitable for real-time applications like chat systems, online gaming, and recommendation engines.

5. Availability & Fault Tolerance

- Designed for **distributed environments**.
- Data is often replicated across nodes, ensuring **high availability** even if some servers fail.

6. Support for Modern Applications

- Web apps, mobile apps, IoT, AI/ML systems require:
 - Rapid development
 - Flexible data models
 - Massive scalability
- NoSQL fits these requirements better than traditional databases.

Differences Between NoSQL and SQL (Relational) Databases

Feature	NoSQL Databases	SQL (Relational) Databases
Data Model	Non-tabular (documents, key-value, graphs)	Tabular (rows & columns in tables)
Schema	Dynamic (schemaless)	Fixed (predefined schema)
Scalability	Horizontal (distributed across servers)	Vertical (scaling up a single server)
Query Language	No standard (varies by database)	SQL (Structured Query Language)
Transactions	Mostly BASE (Basically Available, Soft state, Eventual consistency)	ACID (Atomicity, Consistency, Isolation, Durability)
Use Cases	Big data, real-time apps, unstructured data	Complex queries, transactions, structured data
Examples	MongoDB, Redis, Cassandra, Neo4j	MySQL, PostgreSQL, Oracle, SQL Server

When to Use NoSQL vs. SQL?

Use NoSQL When:

- Your data is **unstructured or semi-structured** (e.g., JSON, logs, social media data).
- You need **high scalability** (e.g., cloud apps, IoT, big data).
- You require **fast read/write operations** (e.g., real-time analytics).
- Your application has **evolving data models** (no fixed schema).

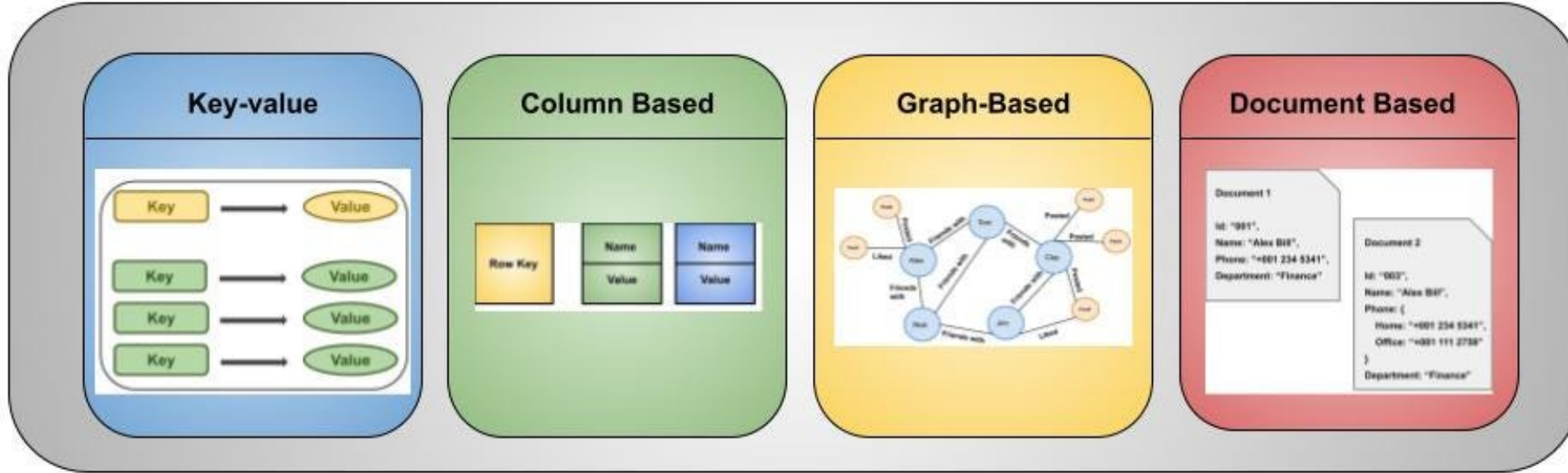
Use SQL When:

- Your data is **highly structured** (e.g., financial records, inventory).
- You need **complex queries** (joins, aggregations).
- **ACID compliance** is critical (e.g., banking systems).
- Your application requires **strict data integrity**.

NoSQL = Flexible, scalable, high performance for unstructured data.

SQL = Structured, ACID-compliant, best for complex transactions.

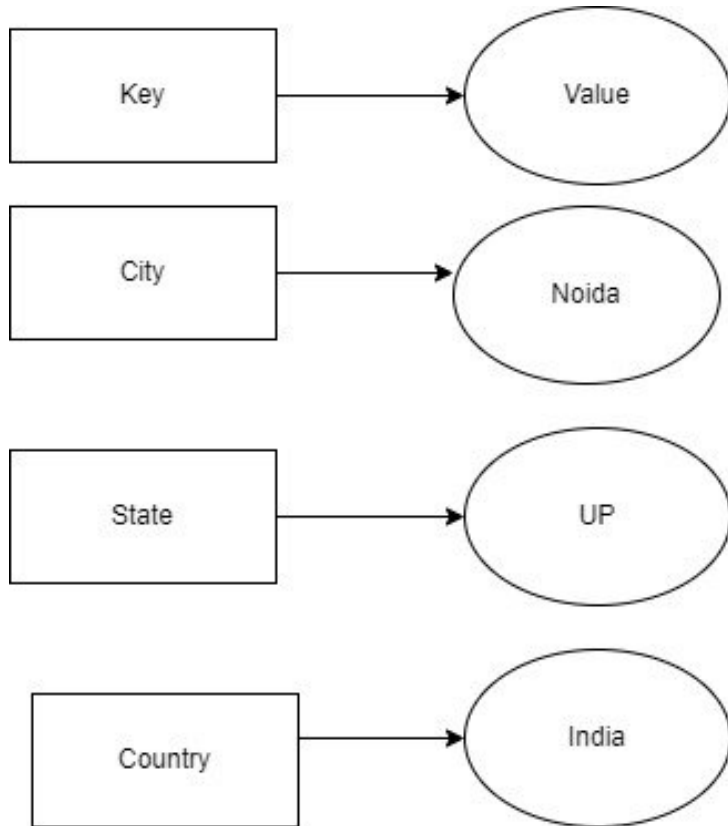
Types of NoSQL Databases / Data Models



Top 10 NoSQL Databases in 2022



1. Key-Value Databases



Description

- *Data is stored as key-value pairs.*
- *Very fast lookup using a unique key.*
- *Simplest NoSQL model.*

Key: "user101"

Value: { "name": "Ram", "age": 21 }

Popular Databases

- Redis
- DynamoDB
- Riak

Use Cases

- Caching
- Session management
- User preferences

Phone directory

Key	Value
Paul	(091) 9786453778
Greg	(091) 9686154559
Marco	(091) 9868564334

MAC table

Key	Value
10.94.214.172	3c:22:fb:86:c1:b1
10.94.214.173	00:0a:95:9d:68:16
10.94.214.174	3c:1b:fb:45:c4:b1

Redis (Key-value Store)

- *Redis is an in-memory key–value NoSQL database known for very high speed.*
- *Data is stored as key → value pairs and mainly used where fast access is required.*

Redis Example

text

Key: user:1

Value: "Ram"

Applications

- Session management
- Caching frequently used data
- Real-time counters and leaderboards
- Chat applications

Redis CRUD Operations

▶ Create

bash

```
SET user:1 "Ram"
```

▶ Read

bash

```
GET user:1
```

▶ Update

bash

```
SET user:1 "Ram Bahadur"
```

▶ Delete

bash

```
DEL user:1
```

2. Document-Oriented Databases

Contacts

CustomerID	ConnId	Name
CBL2016	XYZ987	Joe Smith
CBL2016	SKR007	Sam Smith

Billing

CustomerID	Type	Cardnum	Expiry
CBL2016	visa	5927...	2020-03
CBL2016	master	6273...	2019-11

Customer

CustomerID	Name	DOB
CBL2016	Bob Jones	1980-01-29

Purchases

CustomerID	Item	amt
CBL2016	mac	2823.52
CBL2016	ipad2	623.52

Connections

CustomerID	ConnId	Relation
CBL2016	XYZ987	Brother
CBL2016	SKR007	Father

DocumentKey: **CBL2016**

```
{
  "Name" : "Bob Jones",
  "DOB" : "1980-01-29",
  "Billing" : [
    {
      "type" : "visa",
      "cardnum" : "5927-2842-2847-3909",
      "expiry" : "2020-03"
    },
    {
      "type" : "master",
      "cardnum" : "6273-2842-2847-3909",
      "expiry" : "2019-11"
    }
  ],
  "Connections" : [
    {
      "CustId" : "XYZ987",
      "Relation" : "Brother"
    },
    {
      "CustId" : "PQR823",
      "Relation" : "Father"
    }
  ],
  "Purchases" : [
    { "id":12, item: "mac", "amt": 2823.52 },
    { "id":19, item: "ipad2", "amt": 623.52 }
  ]
}
```

Description

- Data stored as **documents** (JSON / BSON / XML).
- Each document can have a different structure.
- Easy to map with objects in programming languages.

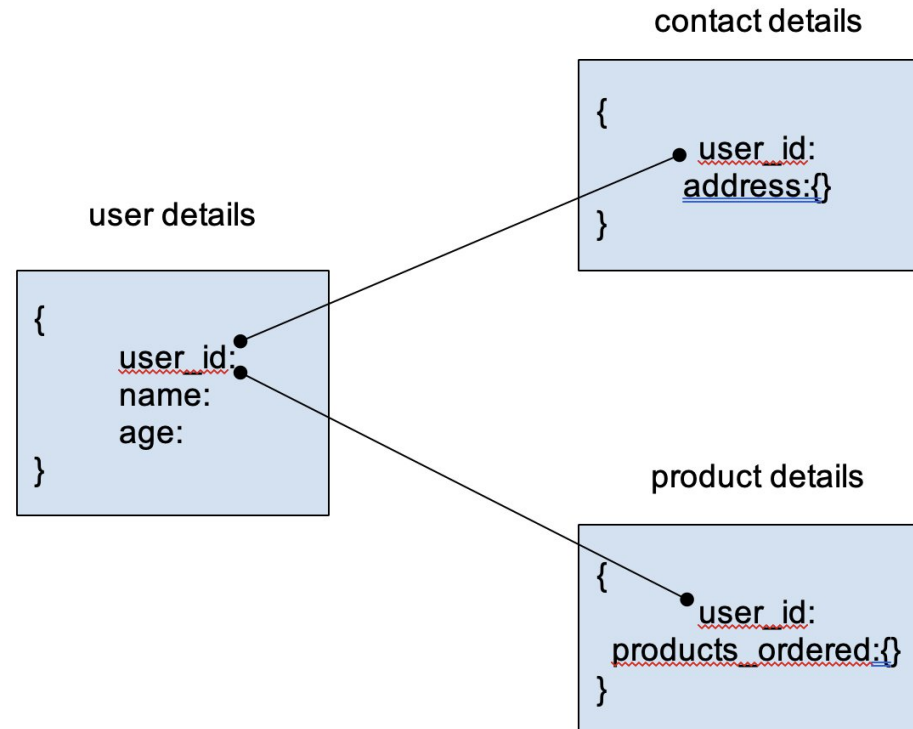
```
{  
  "roll": 101,  
  "name": "Sita",  
  "subjects": ["DBMS", "AI", "Web"]  
}
```

Popular Databases

- MongoDB
- CouchDB
- Firebase Firestore

Use Cases

- Content management systems
- Student records
- E-commerce product catalogs



MongoDB – Document Based NoSQL

MongoDB is a document-oriented NoSQL database that stores data in JSON-like documents (BSON) instead of rows and tables. It is schema-less, highly scalable, and well suited for distributed and web-based applications.

Basic MongoDB Concepts & Syntax

RDBMS Term	MongoDB Term
Database	Database
Table	Collection
Row	Document
Column	Field
Primary Key	<code>_id</code>

CRUD Operations in MongoDB

1 Create (Insert)

Insert one document:

```
db.students.insertOne({  
  name: "Sita",  
  age: 20,  
  faculty: "BSc CSIT"  
})
```

Insert multiple documents:

```
db.students.insertMany([  
  { name: "Ram", age: 21 },  
  { name: "Hari", age: 22 }  
])
```

2 Read (Find)

Find all documents:

```
db.students.find()
```

```
db.students.find({ faculty: "BCA" })
```

3 Update

Update one document:

```
db.students.updateOne(  
  { name: "Ram" },  
  { $set: { age: 22 } }  
)
```

Update many documents:

```
db.students.updateMany(  
  { faculty: "BCA" },  
  { $set: { semester: 6 } }  
)
```

4 Delete

Delete one document:

```
db.students.deleteOne({ name: "Hari" })
```

Delete multiple documents:

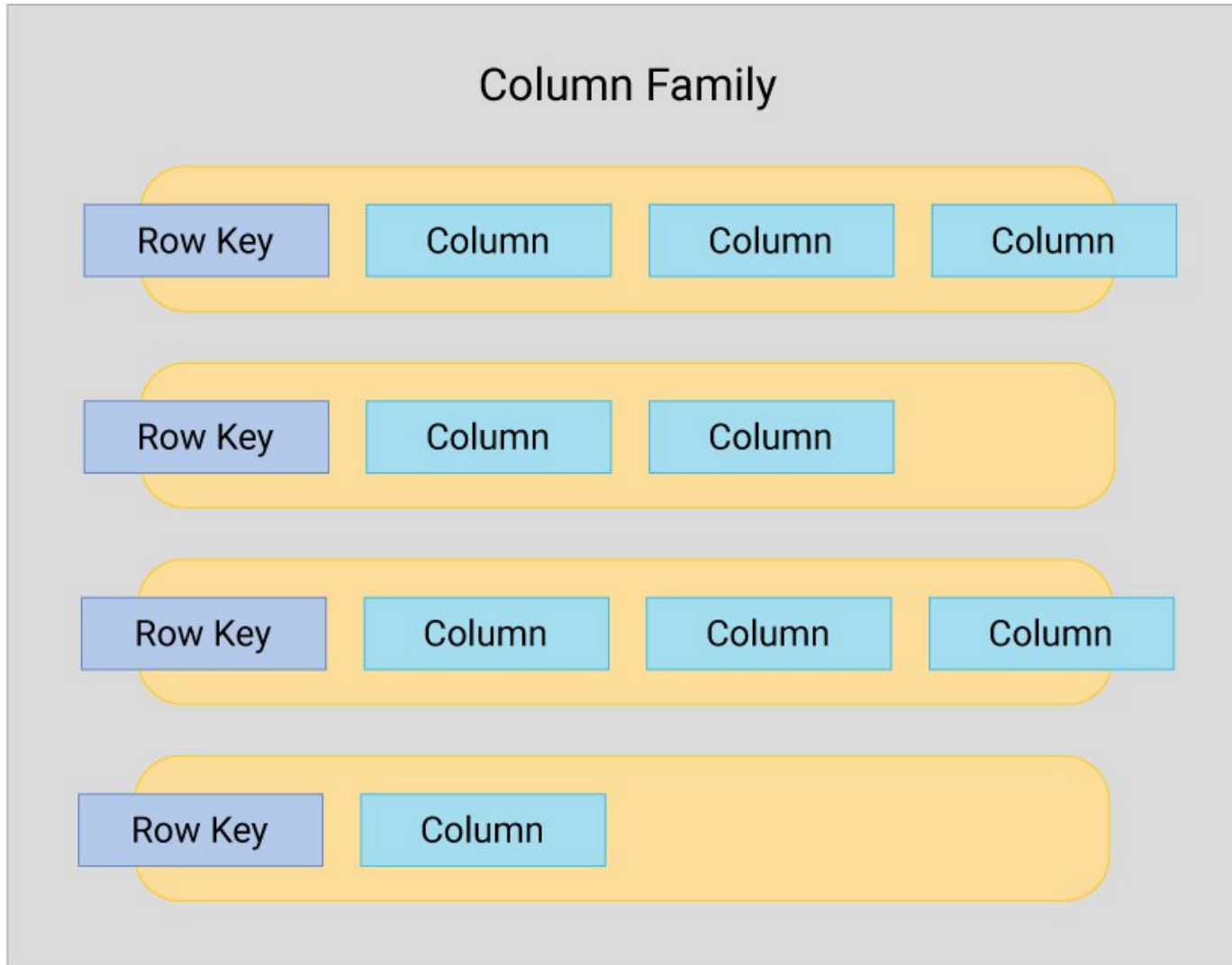
js

```
db.students.deleteMany({ faculty: "BSc CSIT" })
```

Common MongoDB Query Operators

```
{ age: { $gt: 20 } }    // greater than  
{ age: { $lt: 25 } }    // less than  
{ $and: [ {age:20}, {faculty:"BCA"} ] }
```

3. Column-Family Databases



Description

- Data stored in columns instead of rows.
- Columns are grouped into column families.
- Efficient for large-scale read/write operations.

StudentID	Name	Subject	Marks

101	Ram	DBMS	85
101	Ram	AI	90

UserProfile

Bob

emailAddress

bob@example.com

1465676582

gender

male

1465676582

age

35

1465676582

Britney

emailAddress

brit@example.com

1465676432

gender

female

1465676432

Tori

emailAddress

tori@example.com

1435636158

country

Sweden

1435636158

hairColor

Blue

1465633654

Popular Databases

- Apache Cassandra
- HBase

Use Cases

- Data analytics
- Time-series data
- Logging systems

Apache Cassandra (Column-Family Store)

Cassandra is a distributed, column-oriented NoSQL database designed for high availability, fault tolerance, and massive scalability.

It uses CQL (Cassandra Query Language) similar to SQL.

Cassandra Table Example

text

id	name	faculty
1	Sita	BCA

1 | Sita | BCA

Applications

- Big data analytics
- Time-series data (sensor data)
- Banking and transaction systems
- Social media platforms

Cassandra CRUD Operations (CQL)

▶ Create (Insert)

sql

```
INSERT INTO students (id, name, faculty)
VALUES (1, 'Sita', 'BCA');
```

▶ Read

sql

```
SELECT * FROM students;
```

▶ Update

sql

```
UPDATE students  
SET faculty = 'BSc CSIT'  
WHERE id = 1;
```

▶ Delete

sql

```
DELETE FROM students WHERE id = 1;
```

CQL looks like SQL only for ease of learning; internally Cassandra is a distributed, partitioned database where data is organized for fast reads/writes, not relational integrity.

SQL (Join allowed):

sql

```
SELECT * FROM students s  
JOIN courses c ON s.id = c.sid;
```

Cassandra (No Join):

sql

```
SELECT * FROM student_courses WHERE student_id = 1;
```

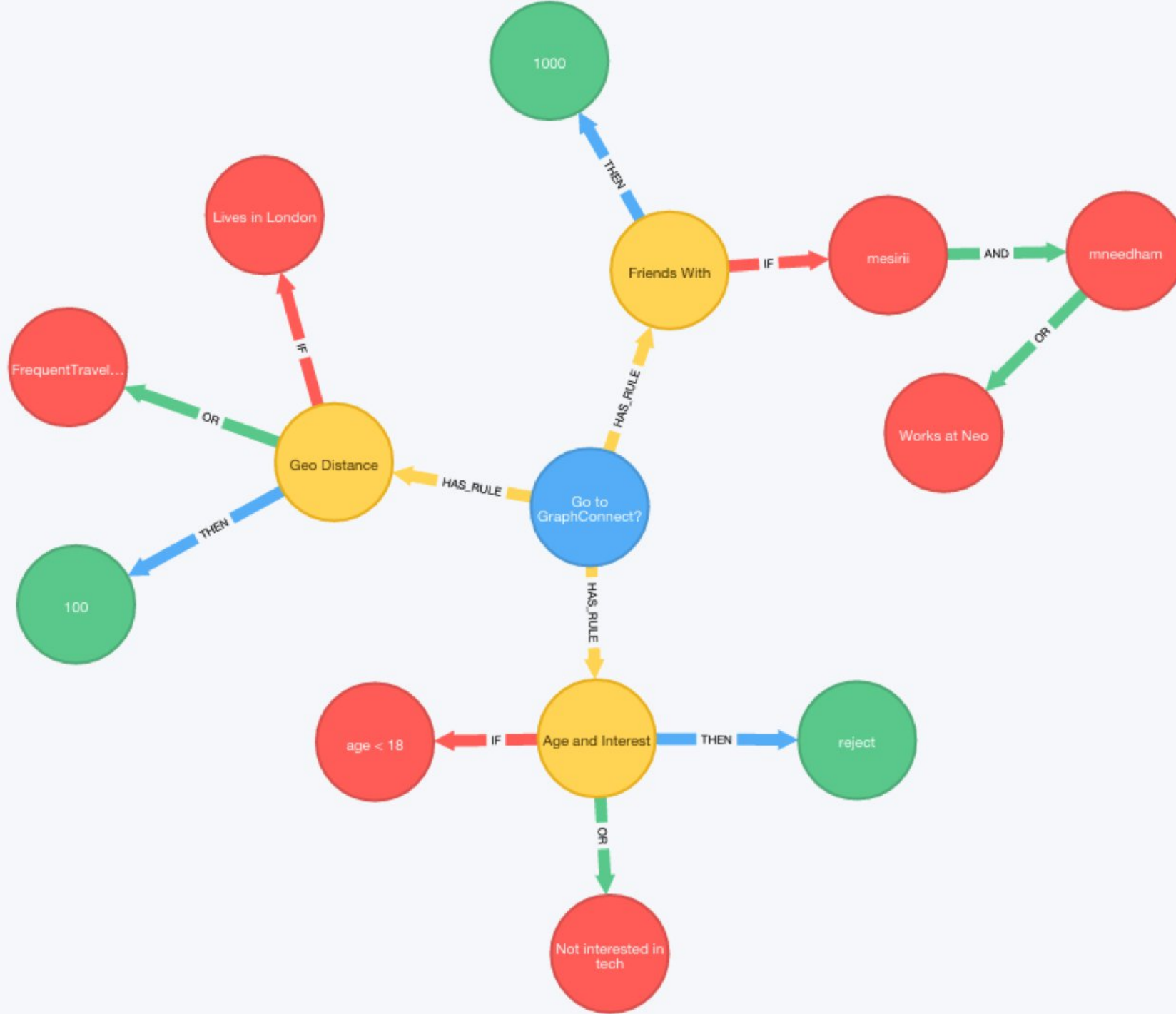
Aspect	SQL (RDBMS)	Cassandra (CQL)
Data Model	Relational (tables with relations)	Column-family (wide-column)
Schema Design	Normalized	Denormalized (query-based design)
Joins	Supports JOINS	✗ No JOINS
Transactions	ACID transactions	Limited (eventual consistency)
Primary Key	Simple primary key	Partition key + clustering key
Scalability	Vertical scaling	Horizontal scaling (distributed)
Consistency	Strong consistency	Tunable consistency
Use Case	Banking, ERP	Big data, time-series

4. Graph Databases

Description

- Data stored as nodes, edges, and properties.
- Focuses on relationships between data.
- Excellent for complex relationships.

Neo4j is a graph-based NoSQL database that stores data as nodes, relationships, and properties. It uses Cypher Query Language.



Popular Databases

- Neo4j
- Amazon Neptune

```
(Ram) -[FRIEND]→ (Sita)
(Ram) -[ENROLLED_IN]→ (DBMS)
```

Use Cases

- Social networks
- Recommendation systems
- Network analysis

Neo4j CRUD Operations (Cypher)

▶ Create (Node)

cypher

```
CREATE (:Student {name:'Ram', faculty:'BCA'})
```

▶ Delete

cypher

```
MATCH (s:Student {name:'Ram'})
DELETE s;
```

▶ Read

cypher

```
MATCH (s:Student) RETURN s;
```

▶ Update

cypher

```
MATCH (s:Student {name:'Ram'})
SET s.faculty = 'BSc CSIT';
```

Graph Database Implementation (Neo4j Example)

Neo4j is a popular **graph database** that uses **Cypher Query Language (CQL)**.

1. What is a Graph Database?

A **graph database** stores data as **nodes** (entities) and **relationships** (connections between entities). Unlike relational databases (SQL), it prioritizes **relationships** over tables.

Key Components:

- **Nodes** = Entities (e.g., `Person`, `Movie`)
- **Relationships** = Connections (e.g., `ACTED_IN`, `DIRECTED`)
- **Properties** = Attributes (e.g., `name`, `age`)

Why Use a Graph Database?

- ✓ **Traversing relationships is fast** (e.g., "Find all actors who worked with Tom Hanks").
- ✓ **Flexible schema** (unlike rigid SQL tables).
- ✓ **Handles complex, interconnected data** (social networks, recommendation engines).

2. Cypher Query Language (CQL)

Cypher is **Neo4j's query language** (like SQL for graph databases). It uses **ASCII-art syntax** to represent patterns.

Basic Syntax:

- `()` = Node
- `-[]->` = Relationship (with direction)
- `:Label` = Node/Relationship type (e.g., `:Person`)
- `{key: value}` = Properties

Task: Build a Graph Database for Trekking Routes in Nepal

Objective: Create a Neo4j graph to model **trekking routes**, **tourist destinations**, and **recommendations** based on difficulty levels.

Problem Statement

Nepal's tourism board wants a system to:

1. Store **trekking routes** (e.g., Everest Base Camp, Annapurna Circuit).
2. Link them to **cities** (e.g., Kathmandu, Pokhara).
3. Recommend treks based on **difficulty level** (Easy/Medium/Hard).

1. Create Nodes (Trekking Routes & Cities)

```
// Create Trekking Routes
CREATE (:Trek {name: "Everest Base Camp", difficulty: "Hard", duration_days: 12});
CREATE (:Trek {name: "Annapurna Circuit", difficulty: "Medium", duration_days: 14});
CREATE (:Trek {name: "Ghorepani Poon Hill", difficulty: "Easy", duration_days: 5});

// Create Cities
CREATE (:City {name: "Kathmandu"});
CREATE (:City {name: "Pokhara"});
CREATE (:City {name: "Lukla"});
```

2. Add Relationships (Which Trek Starts from Which City?)

```
// Link Treks to Cities
MATCH (t:Trek {name: "Everest Base Camp"}), (c:City {name: "Kathmandu"})
CREATE (t)-[:STARTS_FROM]->(c);

MATCH (t:Trek {name: "Annapurna Circuit"}), (c:City {name: "Pokhara"})
CREATE (t)-[:STARTS_FROM]->(c);

MATCH (t:Trek {name: "Ghorepani Poon Hill"}), (c:City {name: "Pokhara"})
CREATE (t)-[:STARTS_FROM]->(c);
```

3. Query the Graph

Q1: Find All Easy Treks Starting from Pokhara

"Ghorepani Poon Hill"

```
MATCH (t:Trek {difficulty: "Easy"})-[:STARTS_FROM]->(c:City {name: "Pokhara"})
RETURN t.name;
```

Q2: Recommend Medium/Hard Treks

```
MATCH (t:Trek)
WHERE t.difficulty IN ["Medium", "Hard"]
RETURN t.name, t.difficulty;
```

"Everest Base Camp" | "Hard"
"Annapurna Circuit" | "Medium"

Q3: Find Which City is the Gateway to Everest Base Camp?

```
MATCH (t:Trek {name: "Everest Base Camp"})-[:STARTS_FROM]->(c:City)
RETURN c.name;
```

"Kathmandu"

Learn with Neo4j's New "[Get to Know Graph & GenAI](#)" Webinar Series >>

[Aura Login](#) [Partners](#) [Company](#) [Support](#) 🔍



[Products](#) ▾

[Use Cases](#) ▾

[Developers](#) ▾

[Generative AI](#)

[Learn](#) ▾

[Pricing](#)

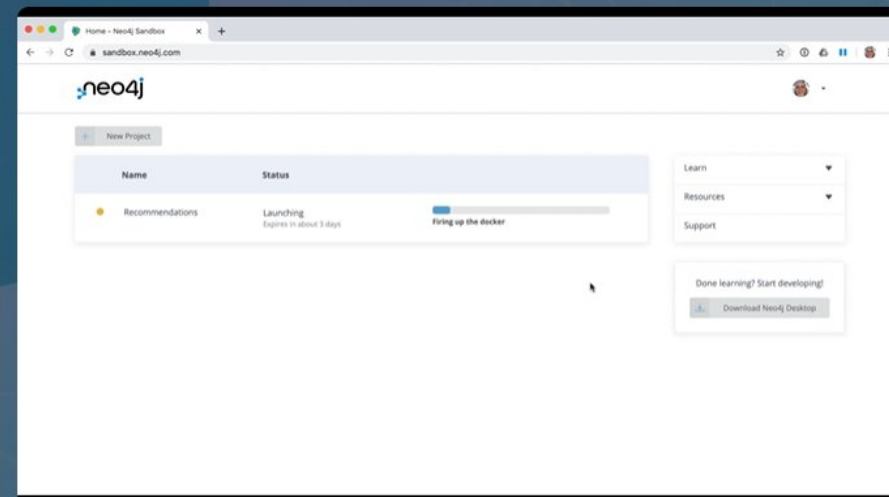
[Contact Us](#)

[Get Started Free](#)

Get started with Neo4j Sandbox while your coffee is still brewing

Neo4j is a native graph database, purpose-built to leverage data relationships and enable richer, more intelligent applications

[Launch the Free Sandbox](#)



[+](#) New Project

Name	Status	
Movies	Running Expires in about 3 days	Open ▼

Actions Connection details Connect via drivers Backups AuraDB Upload Connect via HTTP

Username: neo4j

IP Address: 52.91.1.6

Password: alcohol-pages-dispatches

HTTP Port: 7474

Bolt Port: 7687

Bolt URL: bolt://52.91.1.6:7687

Websocket Bolt URL: bolt+s://eb37bb5ffdcca5a2d480255921325e85.neo4jsandbox.com:7687

Websocket Bolt URL

Get Help

[Neo4j Community](#)

[GraphAcademy \(free online courses\)](#)

[Developer Guides](#)



Master Neo4j with GraphAcademy

Take your Neo4j skills to the next level with free, hands-on courses

[Start Learning](#)

\$ CREATE (:Trek {name: "Everest Base Camp", difficulty: "Hard", duration_day...



neo4j\$ CREATE (:Trek {name: "Everest Base Camp", difficulty: "Hard", du...



neo4j\$ CREATE (:Trek {name: "Annapurna Circuit", difficulty: "Medium", ...



neo4j\$ CREATE (:Trek {name: "Ghorepani Poon Hill", difficulty: "Easy", ...



\$ CREATE (:City {name: "Kathmandu"}); CREATE (:City {name: "Pokhara"}); CREA...



neo4j\$ CREATE (:City {name: "Kathmandu"})



neo4j\$ CREATE (:City {name: "Pokhara"})



neo4j\$ CREATE (:City {name: "Lukla"})



\$ // Link Treks to Cities MATCH (t:Trek {name: "Everest Base Camp"}), (c:Cit...

neo4j\$ MATCH (t:Trek {name: "Everest Base Camp"}), (c:City {name: "Kath...



neo4j\$ MATCH (t:Trek {name: "Annapurna Circuit"}), (c:City {name: "Pokh...



neo4j\$ MATCH (t:Trek {name: "Ghorepani Poon Hill"}), (c:City {name: "Po...



neo4j\$ MATCH (t:Trek {difficulty: "Easy"})-[:STARTS_FROM]→(c:City {name...



Table



Text

t.name

1

"Ghorepani Poon Hill"

```
neo4j$ MATCH (t:Trek) WHERE t.difficulty IN ["Medium", "Hard"] RETURN t...
```



Table



Text



Code

t.name

t.difficulty

1

"Everest Base Camp"

"Hard"

2

"Annapurna Circuit"

"Medium"

```
neo4j$ MATCH (t:Trek {name: "Everest Base Camp"})-[:STARTS_FROM]→(c:Cit...
```



Table



c.name

1

"Kathmandu"

Type	Data Model	Example DB	Best For
Key-Value	Key → Value	Redis	Caching, sessions
Document	JSON docs	MongoDB	Flexible records
Column-Family	Columns	Cassandra	Big data analytics
Graph	Nodes & edges	Neo4j	Relationship data

Aggregation in NoSQL (MongoDB)

Aggregation is a framework for processing data records and returning computed results. It works like a **pipeline** where documents pass through multiple stages, each transforming the data.

Why is it Needed?

- **Data Analysis:** Calculate summaries (sums, averages).
- **Transform Data:** Reshape documents for reporting.
- **Handle Complex Logic:** Multi-step computations.
- **Alternative to Joins:** Since MongoDB is NoSQL, it lacks native joins (but offers alternatives).

2. Key Aggregation Stages

Stage	Purpose	Example
<code>\$match</code>	Filters documents (like <code>WHERE</code>)	<code>{ \$match: { status: "active" } }</code>
<code>\$group</code>	Groups documents (like <code>GROUP BY</code>)	<code>{ \$group: { _id: "\$department", total: { \$sum: "\$salary" } } }</code>
<code>\$lookup</code>	Performs a left-outer-join	<code>{ \$lookup: { from: "orders", localField: "user_id", foreignField: "customer_id", as: "orders" } }</code>
<code>\$unwind</code>	Deconstructs arrays into separate docs ;	<code>{ \$unwind: "\$tags" }</code>
<code>\$project</code>	Reshapes documents (like <code>SELECT</code>)	<code>{ \$project: { name: 1, age: 1 } }</code>

Key Takeaways

- ✓ `1` = Ascending (A-Z, 0-9, lowest to highest)
- ✓ `-1` = Descending (Z-A, 9-0, highest to lowest)

Question 1: Find Total Sales per Product Category

Problem:

Given a `sales` collection, calculate total sales amount for each product category.

Data:

```
db.sales.insertMany([
  { product: "Laptop", category: "Electronics", amount: 1200 },
  { product: "Phone", category: "Electronics", amount: 800 },
  { product: "Shirt", category: "Clothing", amount: 50 },
  { product: "Shoes", category: "Clothing", amount: 100 }
]);
```

Solution:

javascript

```
db.sales.aggregate([
  {
    $group: {
      _id: "$category", // Group by category
      totalSales: { $sum: "$amount" } // Sum amounts
    }
  },
  { $sort: { totalSales: -1 } } // Sort highest to lowest
]);
```

Output:

javascript

```
[
  { "_id": "Electronics", "totalSales": 2000 },
  { "_id": "Clothing", "totalSales": 150 }
]
```

Question 2: Average Age of Users by City

Problem:

Calculate the average age of users grouped by city.

Data:

javascript

```
db.users.insertMany([
  { name: "Alice", age: 25, city: "New York" },
  { name: "Bob", age: 30, city: "New York" },
  { name: "Charlie", age: 22, city: "London" }
]);
```

Output:

javascript

```
[
  { "city": "New York", "avgAge": 27.5 },
  { "city": "London", "avgAge": 22 }
]
```

Solution:

javascript

```
db.users.aggregate([
  {
    $group: {
      _id: "$city",
      avgAge: { $avg: "$age" } // Calculate average
    }
  },
  { $project: { city: "$_id", avgAge: 1, _id: 0 } } // Clean up output
]);
```

Question 3: Most Popular Tags in Blog Posts

Problem:

Find which tags appear most frequently across blog posts (tags are stored as arrays).

Data:

```
db.posts.insertMany([
  { title: "Post 1", tags: ["mongodb", "nosql"] },
  { title: "Post 2", tags: ["javascript", "mongodb"] },
  { title: "Post 3", tags: ["nosql"] }
]);
```

Solution:

javascript

```
db.posts.aggregate([
  { $unwind: "$tags" }, // Split tags into separate docs
  {
    $group: {
      _id: "$tags", // Group by tag
      count: { $sum: 1 } // Count occurrences
    }
  },
  { $sort: { count: -1 } } // Sort by most popular
]);
```

Output:

javascript

```
[
  { "_id": "mongodb", "count": 2 },
  { "_id": "nosql", "count": 2 },
  { "_id": "javascript", "count": 1 }
]
```

3. Advanced Example: Multi-Stage Aggregation

Problem:

Find the top 3 highest-paid employees in each department.

Data:

- `$$ROOT` = Refers to the full document in the pipeline.
- `$push` = Collects all grouped documents into an array.
- `$slice` = Limits array elements (like SQL `LIMIT` but per group).

```
db.employees.aggregate([
  {
    $sort: { department: 1, salary: -1 } // Sort by dept then salary (desc)
  },
  {
    $group: {
      _id: "$department",
      employees: { $push: "$$ROOT" } // Store sorted docs in array
    }
  },
  {
    $project: {
      topEmployees: { $slice: ["$employees", 3] } // Slice first 3 per dept
    }
  }
]);
```

```
db.employees.insertMany([
  { name: "Alice", department: "HR", salary: 60000 },
  { name: "Bob", department: "HR", salary: 75000 },
  { name: "Charlie", department: "Engineering", salary: 90000 },
  { name: "David", department: "Engineering", salary: 110000 }
]);
```

Output:

javascript

```
[
  {
    "_id": "HR",
    "topEmployees": [
      { "name": "Bob", "department": "HR", "salary": 75000 },
      { "name": "Alice", "department": "HR", "salary": 60000 }
    ]
  },
  {
    "_id": "Engineering",
    "topEmployees": [
      { "name": "David", "department": "Engineering", "salary": 110000 },
      { "name": "Charlie", "department": "Engineering", "salary": 90000 }
    ]
  }
]
```

Sharding in NoSQL (Horizontal Scaling)

- *Sharding is a technique used in NoSQL databases to split large datasets into smaller parts (called shards) and distribute them across multiple servers (nodes).*
- *Each shard stores only a portion of the total data, but together they represent the complete database.*

👉 *Purpose: Improve scalability, performance, and availability.*

Sharding is conceptually similar to horizontal fragmentation, but in NoSQL systems data is automatically partitioned into shards rather than physically stored as separate tables as in RDBMS.

Aspect	Horizontal Fragmentation (RDBMS)	Sharding (NoSQL)
Used In	Distributed RDBMS	NoSQL databases
Data Unit	Tables (rows)	Collections / partitions
Schema	Fixed schema	Schema-less / flexible
Joins	Supported	Usually not supported
Control	DBA-defined	Automatic / system-managed
Scaling	Limited	Massive horizontal scaling

Suppose a college has 1,00,000 student records.

Without Sharding

- All records stored on **one server**
- Server becomes slow as data grows

Shard Key

- A **shard key** decides how data is distributed
- Example: `rollNo` , `userId` , `orderId`

With Sharding

Data is divided across **multiple servers** based on a **shard key**.

Shard	Server	Data Stored
Shard 1	Server A	Roll No 1 – 30000
Shard 2	Server B	Roll No 30001 – 60000
Shard 3	Server C	Roll No 60001 – 100000

➔ If you search for Roll No 45000, the system directly checks Server B only.

Sharding in MongoDB (Sharding Strategies)

In MongoDB, sharding means distributing a collection's documents across multiple shards using a shard key. MongoDB supports three main sharding strategies.

Range-Based Sharding

How it Works

- Documents are distributed based on **ranges of shard key values**.
- Similar values go to the same shard.

Shard key: `rollNo`

Shard	Data Stored
Shard 1	rollNo 1 – 3000
Shard 2	rollNo 3001 – 6000
Shard 3	rollNo 6001 – 9000

```
{ "rollNo": 4500, "name": "Ram" }
```

➡ Stored in **Shard 2**

Advantage

- Efficient for **range queries**

js

```
db.students.find({ rollNo: { $gte: 4000, $lte: 5000 } })
```

Disadvantage

- Risk of **hotspot** if values increase sequentially

The hotspot problem occurs in a sharded database when too many read/write operations are directed to a single shard, while other shards remain underutilized.

This usually happens when:

- The shard key values increase sequentially (e.g., auto-increment ID, timestamp)
- Data is range-based sharded

Simple Example (Hotspot)

Shard key: `orderId` (increasing)

Shard	Range	Load
Shard 1	1–1000	Low
Shard 2	1001–2000	Low
Shard 3	2001–3000	Very High

➡ All new inserts go to **Shard 3** → hotspot

The hotspot problem occurs when one shard receives most of the workload due to sequential shard keys, and hash-based sharding avoids it by evenly distributing data using a hash function.

2 Hash-Based Sharding

How it Works

- MongoDB applies a **hash function** on the shard key.
- Hashed values are evenly distributed.

Example

Shard key: `studentId`

studentId	Hashed Value	Shard
101	$h(101)$	Shard A
102	$h(102)$	Shard C
103	$h(103)$	Shard B

```
sh.shardCollection("collegeDB.students", { studentId: "hashed" })
```

Advantage

- Uniform data distribution
- Avoids hotspot problem

Disadvantage

- Inefficient for range queries

3 Zone-Based Sharding (Tag-Aware Sharding)

How it Works

- Shards are grouped into **zones (tags)**.
- Data routed based on shard key ranges or values.

Example

Shard key: `province`

Zone	Shards	Data
East	Shard 1	Province 1, 2
West	Shard 2	Province 5, 6

```
{ "name": "Sita", "province": "Bagmati" }
```

➔ Stored in **East Zone**

Advantage

- Geographic data locality
- Compliance and performance optimization

Graph Traversal Algorithms

- *Graph traversal algorithms are techniques used to visit (explore) all vertices (nodes) and edges of a graph in a systematic way.*
- *They are fundamental in graph databases (Neo4j), AI, network analysis, and DBMS.*

Why Graph Traversal is Needed

- To search data in graphs
- To find paths or connections
- To analyze relationships
- To detect cycles or connectivity

1 Breadth First Search (BFS)

Explanation

- Visits nodes **level by level**
- Uses a **Queue (FIFO)**

Steps

1. Start from a source node
2. Visit all immediate neighbors
3. Then move to next level neighbors

Applications

- Shortest path in **unweighted graphs**
- Social network friend suggestions
- Web crawling

Time Complexity

- $O(V + E)$

2 Depth First Search (DFS)

Explanation

- Goes **deep into the graph** before backtracking
- Uses **Stack** or **Recursion**

Steps

1. Start from a node
2. Visit one neighbor fully
3. Backtrack and explore others

Applications

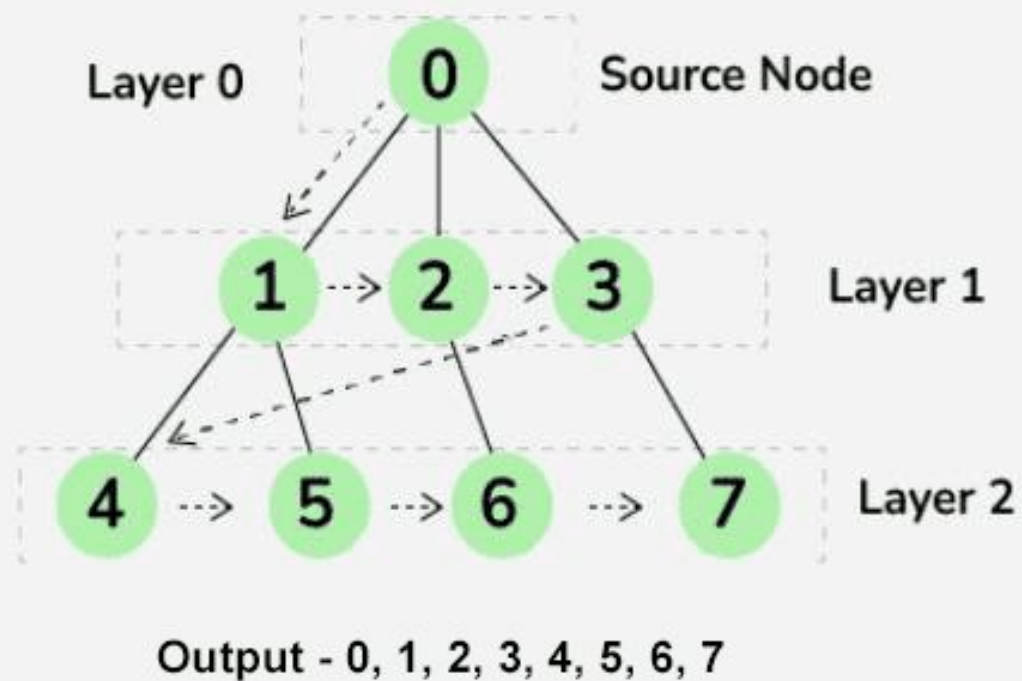
- Cycle detection
- Topological sorting
- Path existence checking

Time Complexity

- $O(V + E)$

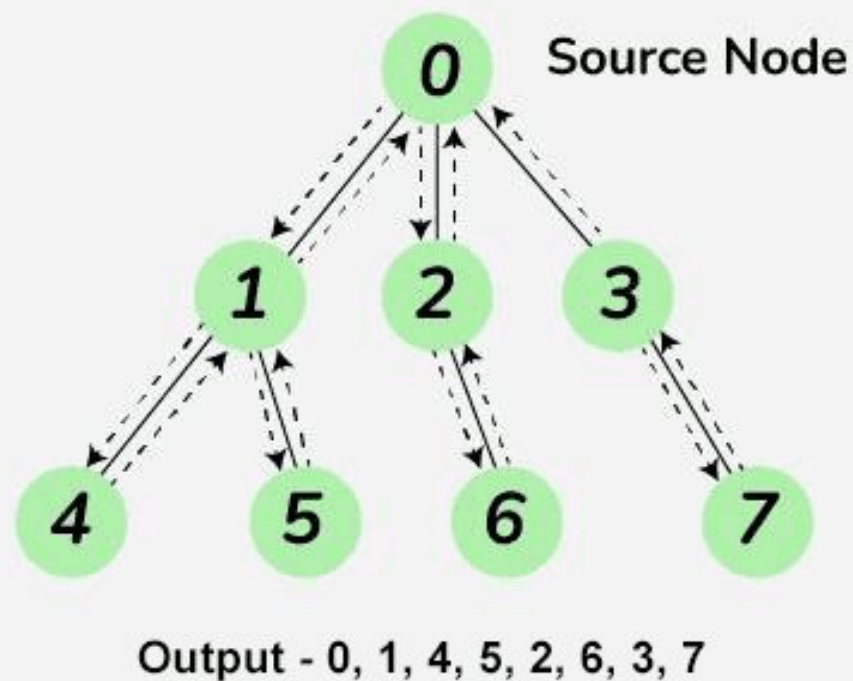


BFS



VS

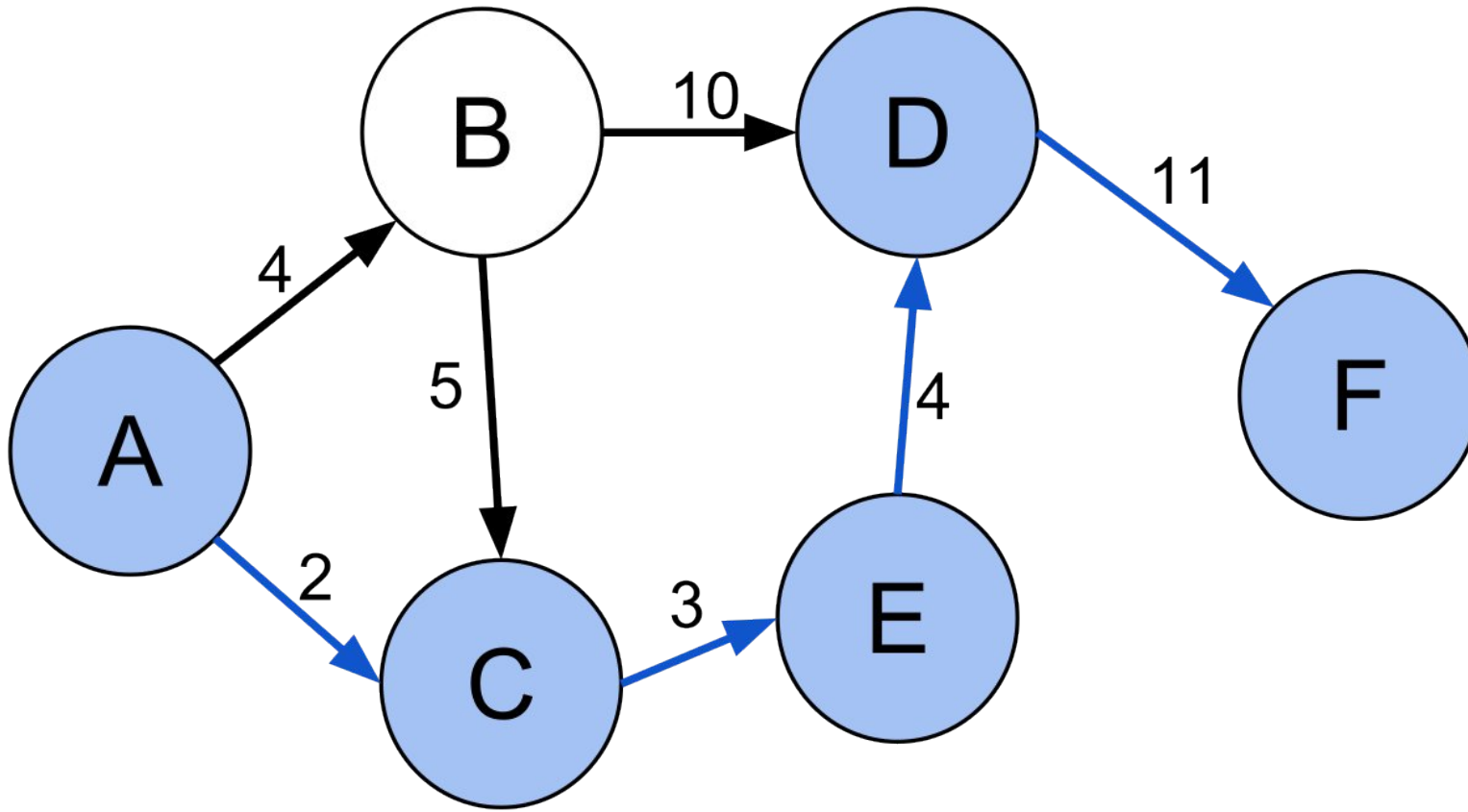
DFS



Difference Between BFS and DFS



3 Dijkstra's Algorithm



Explanation

- *Finds shortest path in weighted graphs*
- *Uses priority queue*

Applications

- *GPS navigation*
- *Network routing*
- *Cost optimization*

A* Search Algorithm

Dijkstra's algorithm

$$f(n) = g(n) + h(n)$$

$f(n)$: the estimate of the **total cost** from **start node** to **target node** through node n

$g(n)$: **actual cost** from **start node** to **node n**

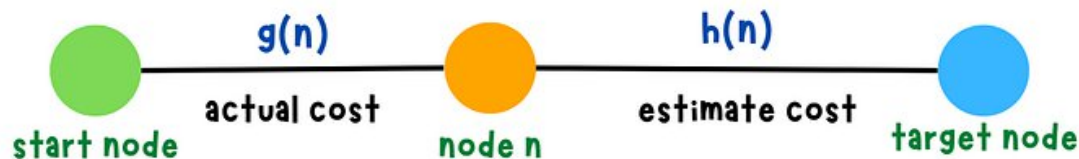
$h(n)$: **estimated cost** from **node n** to **target node**

heuristic function

not actually reach
the target node



educated guess



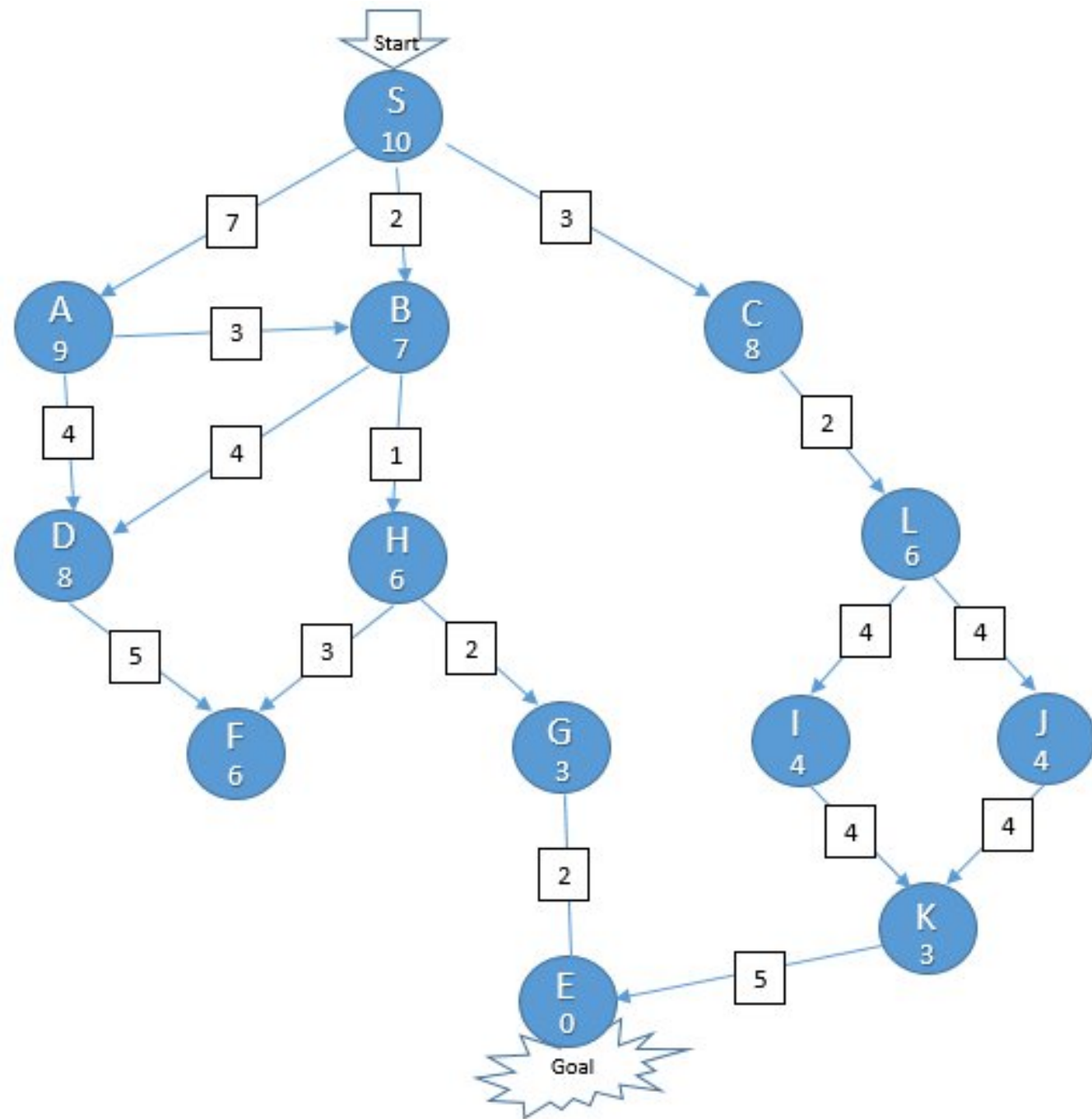
4 A* (A-Star) Algorithm

Explanation

- *Extension of Dijkstra*
- *Uses heuristic function to guide search*

Applications

- *Game pathfinding*
- *AI search problems*
- *Robotics*



Step 6 (a) - Move the first node in the OPEN list to the CLOSED list and expand its immediate successors by adding them to the OPEN list

OPEN				CLOSED	
Node	$g(n)$	$h(n)$	$f(n)$	Node	Parent Node
C	3	8	11	S	
F	6	6	12	B	S
D	6	8	14	H	B
A	7	9	16	G	H
				E	G

EXIT returning 'True' as the Goal node (E) is moved to the CLOSED list. Backtrack the closed list to get the optimal path (E → G → H → B → S)

Graph Traversal in Graph Databases (Neo4j)

Example (Cypher):

cypher

```
MATCH (a)-[:FRIEND*1..3]->(b)
RETURN a, b;
```

➡ Traverses relationships up to **depth 3**

Step 1: Create Person Nodes

cypher

```
CREATE (:Person {name:'Ram'});
CREATE (:Person {name:'Sita'});
CREATE (:Person {name:'Hari'});
```

Scenario

Students in a college are connected as **friends**.

Step 2: Create FRIEND Relationships

cypher

```
MATCH (a:Person {name:'Ram'}), (b:Person {name:'Sita'})
CREATE (a)-[:FRIEND]->(b);

MATCH (b:Person {name:'Sita'}), (c:Person {name:'Hari'})
CREATE (b)-[:FRIEND]->(c);
```

Graph structure:

nginx

Ram → Sita → Hari

Step 3: Read / Traverse the Graph

Find all friends of Ram

cypher

```
MATCH (p:Person {name:'Ram'})-[:FRIEND]->(f)
RETURN f.name;
```

Find friends of friends (2-level traversal)

cypher

```
MATCH (p:Person {name:'Ram'})-[:FRIEND*2]->(f)
RETURN f.name;
```

➡ Output: Hari

Step 4: Update Node Property

cypher

```
MATCH (p:Person {name:'Ram'})
SET p.age = 21;
```

Step 5: Delete Relationship

cypher

```
MATCH (a:Person {name:'Ram'})-[r:FRIEND]->(b:Person {name:'Sita'})
DELETE r;
```

Homework

9. What is Sharding? Explain Sharding Operation and its implementation using any NoSQL database tools.
10. Write the Key features of NoSQL System. Describe how CRUD operations are performed using any NoSQL database tool of your choice. [PU, 2025]
11. Define SQL and NoSQL. Explain different types of NoSQL database with their data model, characteristics, advantages and disadvantages. [PU, 2025]